RAMP-VO Notes

Sebastiano Mengozzi

March 2026

1 Goal and optimization variables

RAMP-VO is a learned visual odometry system that fuses standard images and event stacks in an asynchronous way. Its backend is inherited from DPVO: the algorithm estimates a sliding window of camera poses together with patch depths by minimizing a patch reprojection error. For an active set of frames and patches, the optimization variables are

$$\mathbf{X} = \left\{ \mathbf{T}_i, d_j^l \right\}, \tag{1}$$

where

- $\mathbf{T}_i \in \mathrm{SE}(3)$: camera pose at time index i, mapping world coordinates into camera-i coordinates,
- $d_j^l \in \mathbb{R}$: depth associated with patch l extracted at source frame j.

The backend minimizes

$$J(\mathbf{X}) = \sum_{(l,i)\in\mathcal{E}} \left\| \left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \right] - \hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l) \right\|^2_{\sigma_{li}}, \tag{2}$$

where we use the shorthand

$$\|\mathbf{r}\|^2_{\sigma_{li}} := \mathbf{r}^\top \sigma_{li} \mathbf{r}, \tag{3}$$

for a generic residual vector $\mathbf{r} \in \mathbb{R}^2$ associated with edge (l,i), and define

$$\mathbf{P}'^{l}_{ji} = \omega(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l) \in \mathbb{R}^{4\times p^2}, \qquad \hat{\mathbf{P}}'^{l}_{ji} = \hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l) \in \mathbb{R}^2, \tag{4}$$

Here $\mathbf{P}_j^l \in \mathbb{R}^{4\times p^2}$ denotes patch l extracted from source frame j. The operator $\omega(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l)$ transforms and projects all patch pixels into frame i, while $\hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l)$ applies the same geometric operations and then extracts only the patch center. So the hat means: same reprojection as ω, plus center extraction. Equivalently, without the weighted-norm shorthand,

$$J(\mathbf{X}) = \sum_{(l,i)\in\mathcal{E}} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \right] - \hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l) \right)^\top \sigma_{li} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \right] - \hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l) \right). \tag{5}$$

Using the explicit reprojection map introduced later in Section 5, the same cost can be written even more explicitly as

$$J(\mathbf{X}) = \sum_{(l,i)\in\mathcal{E}} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \right] - \mathrm{center}\left(\bar{\mathbf{K}}\, \mathbf{T}_i \mathbf{T}_j^{-1} \bar{\mathbf{K}}^{-1}\, \mathbf{P}_j^l \right) \right)^\top \sigma_{li} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \right] - \mathrm{center}\left(\bar{\mathbf{K}}\, \mathbf{T}_i \mathbf{T}_j^{-1} \bar{\mathbf{K}}^{-1}\, \mathbf{P}_j^l \right) \right), \tag{6}$$

Here $\mathrm{center}(\cdot)$ means extraction of the patch center, so it is the same operation denoted elsewhere by the hat. Also, $\mathcal{E}$ is the active patch–frame graph, and for each edge $(l,i) \in \mathcal{E}$, the DPVO backend predicts a learned 2D correction $\Delta_{li} \in \mathbb{R}^2$ and a weight matrix $\sigma_{li} \in \mathbb{R}^{2\times 2}$. Thus RAMP-VO still solves a geometric alignment problem, but the correspondence updates $(\Delta_{li}, \sigma_{li})$ are predicted by a learned asynchronous encoder and a learned update operator from the DPVO backend.

2 Asynchronous input stream

The paper uses the word *frame* for both ordinary images and event stacks. The input is a temporally ordered asynchronous stream

$$\left\{\mathbf{I}_{k_j}(t_j)\right\}_{j=1}^{T}, \tag{7}$$

where

- $k_j \in \{\mathrm{e}, \mathrm{i}\}$ indicates whether the sample at time t_j comes from the event sensor or the image sensor,
- $\mathbf{I}_{\mathrm{e}}(t_j) \in \mathbb{R}^{5\times H\times W}$ is an event stack,
- $\mathbf{I}_{\mathrm{i}}(t_j) \in \mathbb{R}^{C\times H\times W}$ is an image.

The key modeling choice is that these inputs are *not* forced onto one common synchronous rate. Instead, every time a new image or event stack arrives, the internal representation is updated. This is the main difference from a naive "concatenate images and events" pipeline.

3 High-level decomposition

For each incoming element of the asynchronous stream, RAMP-VO performs four conceptual steps:

1. encode the new image/event input with the RAMP encoder to produce matching and context features,
2. extract patches in informative regions,
3. connect those patches to nearby frames and estimate projection corrections,
4. run differentiable bundle adjustment to update poses and patch depths.

Pose forecasting is then used to initialize the next pose estimate more accurately than a simple constant-velocity extrapolation.

4 Patch representation

Like DPVO, RAMP-VO does not optimize sparse keypoints as single pixels. It optimizes small image patches. For patch l extracted from source frame j, the dimensionally consistent interpretation is

$$\mathbf{P}_j^l = \begin{bmatrix}\mathbf{x}\\ \mathbf{y}\\ \mathbf{1}\\ \mathbf{d}\end{bmatrix} \in \mathbb{R}^{4\times p^2}, \qquad \mathbf{x}, \mathbf{y}, \mathbf{1}, \mathbf{d} \in \mathbb{R}^{1\times p^2}. \tag{8}$$

Here:

- $\mathbf{x}, \mathbf{y}$ store the pixel coordinates of all p^2 pixels in the patch,
- $\mathbf{1}$ is a row of ones used for homogeneous coordinates,
- $\mathbf{d}$ stores the depth values, taken constant within the patch in the implementation described by the paper.

So each column of $\mathbf{P}_j^l$ corresponds to one pixel of the patch written in augmented coordinates. The paper writes this object in a compact transposed form, but taken literally that notation is ambiguous; the reprojection formula only works dimension-wise when $\mathbf{P}_j^l$ is interpreted as a $4 \times p^2$ matrix.

The patch centers are chosen from regions with high event density:

- count events per pixel to build a density map,
- apply non-maximum suppression over an 11×11 neighborhood,
- keep the N strongest surviving locations.

This is a practical design choice: if patch centers were sampled without considering event activity, many would fall in regions where events provide little or no tracking information.

5 Patch reprojection graph

Each source patch is connected to nearby frames within a temporal index radius r, meaning that

$$|i - j| \leq r. \tag{9}$$

So r is a user-chosen backend hyperparameter that controls how many past/future neighboring frames are connected to a patch in the graph. This creates a bipartite graph $\mathcal{E}$ between source patches and target frames. Let $\mathbf{K} \in \mathbb{R}^{3\times 3}$ denote the camera intrinsic calibration matrix. The projection of patch l from source frame j into frame i is

$$\mathbf{P}'^l_{ji} \sim \bar{\mathbf{K}}\, \mathbf{T}_i \mathbf{T}_j^{-1} \bar{\mathbf{K}}^{-1}\, \mathbf{P}_j^l, \qquad \bar{\mathbf{K}} = \begin{pmatrix} \mathbf{K} & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \tag{10}$$

In words, this multiplication first lifts the source patch from image coordinates into camera coordinates through $\bar{\mathbf{K}}^{-1}$, then transports it from frame j to frame i through the relative pose $\mathbf{T}_i \mathbf{T}_j^{-1}$, and finally maps it back to image coordinates through $\bar{\mathbf{K}}$. The paper abbreviates this as

$$\mathbf{P}'^l_{ji} = \omega(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}_j^l). \tag{11}$$

Conceptually, this is the state-implied geometric prediction: if the current poses and depth are correct, the patch should appear at this projected location.

6 Learned correspondence update

For every active edge $(l, i) \in \mathcal{E}$, RAMP-VO compares the source patch features with the target-frame features and predicts:

- a 2D correction Δ_{li} telling how the current projection should move in the image plane,
- a confidence matrix σ_{li} telling how strongly this correction should influence optimization.

The paper summarizes the full correlation and refinement stack of the DPVO backend by the update operator

$$\Delta_{li}, \sigma_{li} = F(\mathbf{P}'^l_{ji}, \mathbf{c}_i, \mathbf{m}_i, \mathbf{m}_j), \tag{12}$$

where

- $\mathbf{m}_j$ are matching features,
- $\mathbf{c}_i$ are context features.

The important point is that the learned frontend does *not* directly output the full pose. Instead, it outputs patch-level corrections that are consumed by a geometric bundle-adjustment layer. This keeps the final pose estimation tied to multi-view geometry.

7 Differentiable bundle adjustment backend

The backend solves Equation (2). It tries to make the corrected patch observations

$$\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li} \tag{13}$$

match the reprojection predicted by the current poses and patch depths

$$\hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}^l_j). \tag{14}$$

The optimization updates

- the camera poses in the current window,
- the patch depths.

As in DPVO, the paper uses a differentiable bundle-adjustment (BA) layer that performs a small number of damped nonlinear least-squares updates, in the spirit of Levenberg–Marquardt, on the pose and depth variables of Equation (2). "Differentiable" means that, during training, gradients are backpropagated through these optimizer updates as well.

Intuitively, the learned network says *where* each patch should move, and the BA layer finds the set of poses and depths that best explains all those corrected tracks jointly.

8 RAMP encoder: why it is different

The central contribution of the paper is the encoder that fuses asynchronous event and image data. The encoder maintains recurrent multi-scale sensor-agnostic states and updates them whenever new data arrive. The processing is split into three stages.

8.1 Stage 1: sensor-specific pixel-wise encoders

For each scale $s \in \{0, 1, 2\}$, where s is the index of the spatial resolution at which the same image/event input is processed ($s = 0$ full resolution, $s = 1$ half resolution, $s = 2$ quarter resolution because the stride is 2^s), the new input is passed through a sensor-specific encoder:

$$\mathbf{f}^s_{k_j}(t_j) = E^s_{k_j}(\mathbf{I}_{k_j}(t_j)). \tag{15}$$

The paper uses three scales with kernel sizes 1×1, 3×3, and 5×5, and stride 2^s. Thus the algorithm extracts local features at several receptive fields while remaining computationally simple.

8.2 Stage 2: intra-sensor recurrent update

Within each sensor stream, features are updated by a pixel-wise LSTM:

$$\mathbf{h}^s_{k_j}(t_j) = G^s_{k_j}(\mathbf{f}^s_{k_j}(t_j)). \tag{16}$$

This step gives each sensor its own temporal memory. Because the recurrence is pixel-wise rather than based on expensive dense ConvGRU blocks, the encoder is much more parallelizable.

8.3 Stage 3: inter-sensor asynchronous fusion

The sensor-specific hidden state is then fused with the shared state from the previous time step:

$$\mathbf{\Sigma}_j^s = H_{k_j}\big([\mathbf{h}_{k_j}^s(t_j) \,\|\|\, \mathbf{\Sigma}_{j-1}^s]\big). \tag{17}$$

This is the crucial asynchronous fusion step. A different depth-wise convolution H_{k_j} is used depending on whether the arriving measurement is an image or an event stack. Therefore:

- the hidden state can be updated at the image rate,
- it can also be updated between images whenever events arrive,
- both modalities contribute to the same latent representation.

This is the mechanism that lets RAMP-VO exploit event timing without resampling everything to a fixed schedule.

9 Multi-scale fusion into DPVO features

After the recurrent states $\{\mathbf{\Sigma}_j^s\}_{s=0}^2$ are available, a hierarchical multi-scale fusion module produces the features used by the DPVO backend:

$$\mathbf{z}_j = K\big(\{\mathbf{\Sigma}_j^s\}_{s=0}^2\big), \tag{18}$$

where $\mathbf{z}_j$ is instantiated either as

$$\mathbf{z}_j = \mathbf{m}_j \qquad \text{or} \qquad \mathbf{z}_j = \mathbf{c}_j. \tag{19}$$

So the encoder ultimately outputs two feature maps:

- matching features $\mathbf{m}_j$, used for correlation and track refinement,
- context features $\mathbf{c}_j$, used by the update operator.

The paper's Multi-Scale Fusion block follows the DPVO feature extractor design, but injects the recurrent multi-scale states partway through the hierarchy. This lets coarse and fine information both influence the tracking features.

10 Pose forecasting

A second paper contribution is pose forecasting for better initialization. Instead of initializing the next pose with a simple linear rule, RAMP-VO uses recent patch tracks. For each patch center, two cubic univariate splines are fitted:

$$\big(x_j^l, y_j^l\big) \sim \mathbf{S}^l(t_j) = \big(S_x^l(t_j), S_y^l(t_j)\big). \tag{20}$$

The splines model the image-plane motion of the patch center over the recent history. The future position at time t_{j+1} is predicted by evaluating

$$\mathbf{S}^l(t_{j+1}). \tag{21}$$

The paper states that the first and second derivatives are assumed zero at the spline boundaries, and the patch depth is assumed constant during this extrapolation. Then BA is run again, now using forecasted patch centers, to recover a forecasted 6-DoF pose at the future timestamp.

This matters because events provide information in the blind time between images, so a future pose estimate can be initialized using actual recent motion trends instead of only the last two camera poses.

11 What is learned and what is geometric

RAMP-VO is not a pure regression model and not a pure classical VO model either. Its structure is hybrid:

- **learned part:** asynchronous multimodal feature extraction, track correction prediction, weighting,
- **geometric part:** patch reprojection model and bundle adjustment over poses and depths.

This split is important for understanding the method. The paper does *not* ask the network to directly emit a trajectory from raw events and frames. Instead, it learns the hard frontend pieces that classical pipelines struggle with in low texture, low light, and asynchronous settings, while keeping geometric consistency in the backend.

12 Why events help in this architecture

The paper's logic is:

- standard images provide dense appearance information,
- events provide high temporal resolution and robustness to blur and HDR conditions,
- asynchronous recurrent fusion lets the latent state keep evolving between image arrivals,
- this improves patch tracking and pose initialization when image rate is low or motion is fast.

Therefore, the contribution is not just *using* events, but using them at their natural asynchronous rate inside the representation itself.

13 Pseudo-algorithm

Algorithm 1 Asynchronous RAMP-VO Update

Input:
previous shared states $\{\mathbf{\Sigma}^s_{j-1}\}^2_{s=0}$
current pose/depth window $\mathbf{X}_{j-1}$
new asynchronous measurement $\mathbf{I}_{k_j}(t_j)$, with $k_j \in \{\mathrm{e}, \mathrm{i}\}$
active patch graph radius r

Update step j:

1. Asynchronous Encoding

for each scale $s \in \{0, 1, 2\}$ **do**
  $\mathbf{f}^s_{k_j}(t_j) \leftarrow E^s_{k_j}(\mathbf{I}_{k_j}(t_j))$
  $\mathbf{h}^s_{k_j}(t_j) \leftarrow G^s_{k_j}(\mathbf{f}^s_{k_j}(t_j))$
  $\mathbf{\Sigma}^s_j \leftarrow H_{k_j}([\mathbf{h}^s_{k_j}(t_j) \,\|\|\, \mathbf{\Sigma}^s_{j-1}])$

$(\mathbf{m}_j, \mathbf{c}_j) \leftarrow K(\{\mathbf{\Sigma}^s_j\}^2_{s=0})$

2. Patch Extraction

Build an event-density map and extract N patch centers with non-maximum suppression
Form source patches $\mathbf{P}^l_j$

3. Graph Construction

Connect each source patch to nearby frames satisfying $|i - j| \leq r$
Build the active bipartite graph $\mathcal{E}$

4. Learned Track Refinement

for each active edge $(l, i) \in \mathcal{E}$ **do**
  Compute geometric reprojection $\mathbf{P}'^l_{ji} = \omega(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}^l_j)$
  Predict correction and weight $(\Delta_{li}, \sigma_{li}) = F(\mathbf{P}'^l_{ji}, \mathbf{c}_i, \mathbf{m}_i, \mathbf{m}_j)$

5. Geometric Update

$J(\mathbf{X}) \leftarrow \sum_{(l,i)\in\mathcal{E}} \left\| \left[\hat{\mathbf{P}}'^l_{ji} + \Delta_{li}\right] - \hat{\omega}(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}^l_j) \right\|^2_{\sigma_{li}}$

$\mathbf{X}_j \leftarrow \arg\min_{\mathbf{X}} J(\mathbf{X})$ ▷ Differentiable bundle adjustment

6. Pose Forecasting

Fit splines to recent patch-center tracks
Extrapolate the next patch centers and solve for the forecast pose used to initialize the next update

14 Compact block scheme

15 Main takeaways

- RAMP-VO inherits the DPVO philosophy: optimize poses and patch depths with differentiable bundle adjustment.
- The main novelty is the asynchronous recurrent encoder that fuses event stacks and images into a shared latent state.
- The network predicts patch-level corrections and confidences, not the full pose directly.
- Pose forecasting uses spline extrapolation of recent patch tracks to initialize future poses better.
- Events help not only because they add another modality, but because the latent state can be updated between image arrivals.

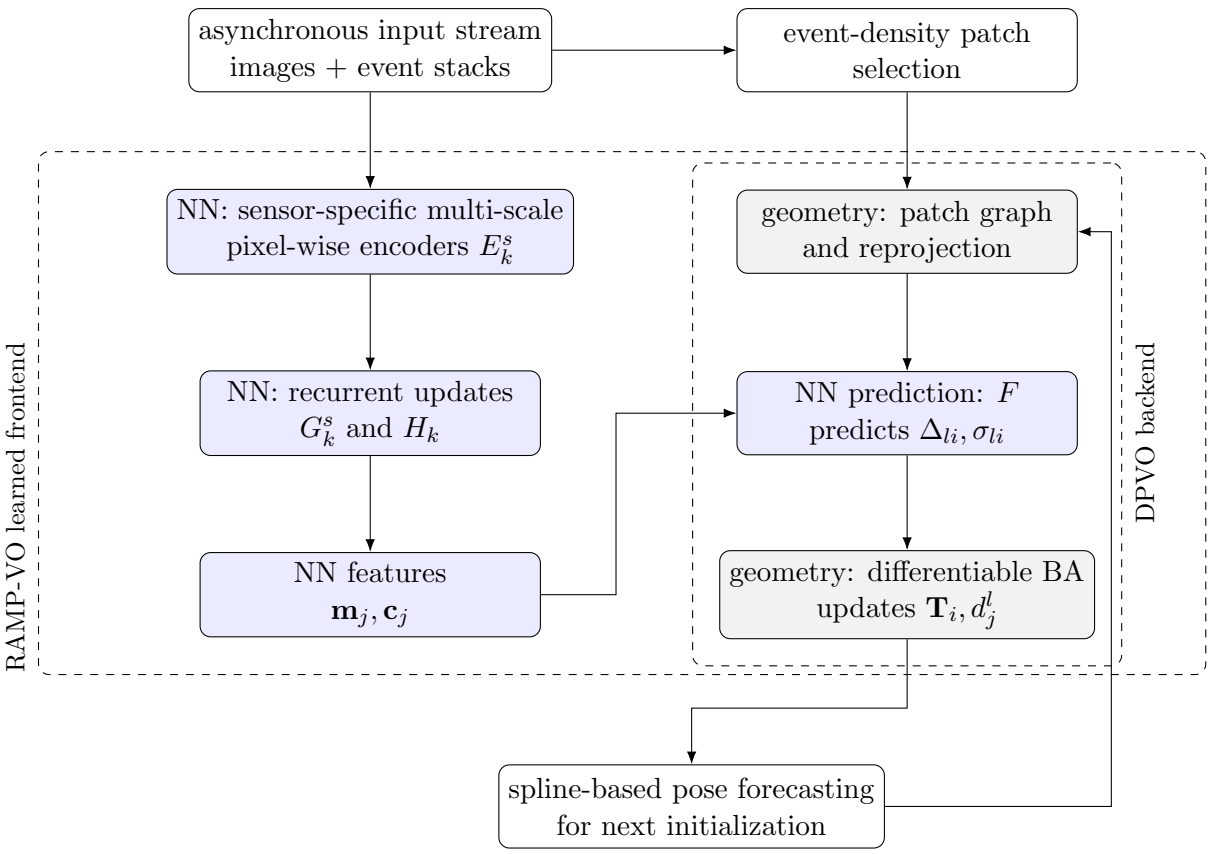

Figure 1: Block decomposition of RAMP-VO. The dashed left group is the RAMP-VO learned frontend, which processes asynchronous images and event stacks and predicts the features $\mathbf{m}_j, \mathbf{c}_j$ together with the per-edge quantities Δ_{li}, σ_{li}. The dashed right group is the DPVO backend inherited from DPVO, which builds reprojection constraints and runs bundle adjustment to update poses and depths.

16 Comparison with the VIO cost

The explicit matrix form of the RAMP-VO backend cost is

$$J(\mathbf{X}) = \sum_{(l,i)\in\mathcal{E}} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li}\right] - \mathrm{center}\Big(\bar{\mathbf{K}}\,\mathbf{T}_i\mathbf{T}_j^{-1}\bar{\mathbf{K}}^{-1}\,\mathbf{P}_j^l\Big)\right)^\top \sigma_{li} \left(\left[\hat{\mathbf{P}}'^{l}_{ji} + \Delta_{li}\right] - \mathrm{center}\Big(\bar{\mathbf{K}}\,\mathbf{T}_i\mathbf{T}_j^{-1}\bar{\mathbf{K}}^{-1}\,\mathbf{P}_j^l\Big)\right). \tag{22}$$

Using the notation of the VIO note, the weighted least-squares cost is

$$J_{\mathrm{VIO}}(\mathbf{X}) = J_{\mathrm{VIO}}(\mathbf{r}_{\mathrm{prior}}, \mathbf{r}_{\mathrm{imu}}) + \sum_{(i,\ell)\in\mathcal{C}} \left(\mathbf{u}_{i\ell} - \mathrm{pix}\left(\mathbf{K}\frac{1}{(\mathbf{P}_\ell^{C_i})_z}\mathbf{P}_\ell^{C_i}\right)\right)^\top \Sigma_{i\ell}^{-1} \left(\mathbf{u}_{i\ell} - \mathrm{pix}\left(\mathbf{K}\frac{1}{(\mathbf{P}_\ell^{C_i})_z}\mathbf{P}_\ell^{C_i}\right)\right), \tag{23}$$

with

$$\mathbf{P}_\ell^{C_i} = \mathbf{R}_{CB}\Big(\mathbf{R}_i^\top(\mathbf{P}_\ell^W - \mathbf{p}_i) - \mathbf{p}_{BC}^B\Big).$$

Structurally, both are weighted sums of quadratic residual terms. The difference is that RAMP-VO uses only patch-reprojection factors over poses and patch depths, with learned corrections Δ_{li} and learned weights σ_{li}, while VIO combines three heterogeneous factor types: a prior

term, IMU factors, and camera reprojection factors. In short, RAMP-VO has one geometrically uniform factor family enriched by neural predictions, whereas VIO merges multiple sensor-model residual families into the same least-squares objective.

A Appendix: technical details

A.1 LSTM Refresher

An LSTM (Long Short-Term Memory) is a recurrent neural-network unit that updates an internal memory state and a hidden state over time. Given an input $\mathbf{u}_t$, previous hidden state $\mathbf{h}_{t-1}$, and previous cell state $\mathbf{c}_{t-1}$, an LSTM computes gates that decide what information to forget, what new information to write, and what part of the internal memory to expose:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{u}_t, \mathbf{h}_{t-1}] + \mathbf{b}_f), \tag{24}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{u}_t, \mathbf{h}_{t-1}] + \mathbf{b}_i), \tag{25}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{u}_t, \mathbf{h}_{t-1}] + \mathbf{b}_c), \tag{26}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{u}_t, \mathbf{h}_{t-1}] + \mathbf{b}_o), \tag{27}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{28}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{29}$$

In RAMP-VO, the paper uses pixel-wise LSTMs, meaning that this recurrent update is applied independently at each pixel location of the feature maps. The role of the LSTM here is to give each sensor stream a temporal memory before the event/image information is fused into the shared state.

A.2 Encoder Refresher

In general, an encoder is a neural module that maps raw inputs into a feature representation that is more useful for the downstream task. In this paper, the raw inputs are images or event stacks, and the encoder produces feature maps that retain motion-relevant information while being easier for the tracking and optimization backend to use. The RAMP encoder is not a single block but a composition of three operations: sensor-specific feature extraction E_k^s, temporal memory update G_k^s, and asynchronous cross-sensor fusion H_k. The output of this encoder is then turned into the matching and context features $\mathbf{m}_j, \mathbf{c}_j$ consumed by the DPVO backend.

A.3 Differentiable Bundle-Adjustment Layer

In this paper, a *differentiable bundle-adjustment layer* is a module that takes the current pose/depth variables, forms the reprojection residuals from Equation (2), and performs a fixed small number of optimization steps to reduce that objective. It is called a *layer* because it is embedded inside the neural network pipeline, and it is called *differentiable* because, during training, gradients are propagated not only through the feature-extraction and update blocks, but also through the optimizer steps themselves. In other words, the loss depends on the optimized variables after BA, and backpropagation differentiates through the computations that produced those optimized variables.

A.4 Damped Nonlinear Least-Squares Updates

The BA objective is nonlinear in the poses and depths, so it is minimized iteratively. Let the stacked residual vector be denoted by

$$\mathbf{r}(\mathbf{X}), \tag{30}$$

with Jacobian

$$\mathbf{J} = \frac{\partial \mathbf{r}}{\partial \mathbf{X}}. \tag{31}$$

At iteration k, one linearizes the residual around the current estimate $\mathbf{X}^{(k)}$, solves for an increment $\delta\mathbf{X}$, and updates the state by

$$\mathbf{X}^{(k+1)} = \mathbf{X}^{(k)} + \delta\mathbf{X}. \tag{32}$$

This additive notation is only a shorthand. In practice, the state contains both Euclidean variables and pose variables on SE(3). If we write

$$\mathbf{X} = \left\{\mathbf{T}_i, d_j^l\right\}, \qquad \delta\mathbf{X} = \left\{\delta\boldsymbol{\xi}_i, \delta d_j^l\right\}, \tag{33}$$

then the effective updates are

$$\mathbf{T}_i^{(k+1)} = \mathrm{Exp}(\delta\boldsymbol{\xi}_i)\,\mathbf{T}_i^{(k)}, \tag{34}$$

$$d_j^{l\,(k+1)} = d_j^{l\,(k)} + \delta d_j^l. \tag{35}$$

In a Levenberg–Marquardt-style step, the increment is obtained from the damped normal equations

$$\left(\mathbf{J}^\top\mathbf{W}\mathbf{J} + \lambda\mathbf{I}\right)\delta\mathbf{X} = -\mathbf{J}^\top\mathbf{W}\mathbf{r}, \tag{36}$$

that is,

$$\delta\mathbf{X} = -\left(\mathbf{J}^\top\mathbf{W}\mathbf{J} + \lambda\mathbf{I}\right)^{-1}\mathbf{J}^\top\mathbf{W}\mathbf{r}. \tag{37}$$

In the present note, $\mathbf{W}$ can be written explicitly as

$$\mathbf{W} = \mathrm{blkdiag}\big(\{\sigma_{li}\}_{(l,i)\in\mathcal{E}}\big), \tag{38}$$

that is, a block-diagonal matrix whose 2×2 blocks are the per-edge confidence matrices σ_{li} from Equation (2). These σ_{li} are predicted by the neural update operator F, together with the corrections Δ_{li}; this learned weighting mechanism is part of the DPVO backend inherited by RAMP-VO. These are called *residual weights* because they determine how strongly each residual contributes to the normal equations: a larger weight means that the corresponding reprojection error has a stronger influence on the update, while a smaller weight downweights that measurement. The scalar $\lambda > 0$ is a damping parameter. When λ is small, the step behaves like Gauss–Newton; when λ is larger, the step becomes more conservative and numerically stable. This is what is meant by "damped nonlinear least-squares updates, in the spirit of Levenberg–Marquardt." In the paper's DPVO backend, only a small fixed number of such updates is run inside the layer.

A.5 Training of the Neural Components

RAMP-VO is trained *end-to-end*: the network runs the full pipeline, produces intermediate trajectory estimates, the loss is computed from these outputs, and gradients are backpropagated through the differentiable modules with the standard training step

```
traj = net(...),    loss = compute_losses(traj,...),    loss.backward().
```
(39)

In this sense, the supervision is applied at the output of the whole system, not by assigning a separate local loss to each block. The trainable neural modules are the RAMP encoder blocks E_k^s, G_k^s, H_k, the multi-scale fusion modules producing $\mathbf{m}_j, \mathbf{c}_j$, and the DPVO backend modules that predict Δ_{li} and σ_{li}. The differentiable BA layer is part of the gradient path, so these losses propagate through the optimizer updates as well.

The practical training setup described by the paper is the following: RAMP-VO is trained on an event-augmented version of TartanAir, following the DPVO training scheme and hyperparameters, for 350,000 steps on sequences of 15 images and 30 event stacks. In the released implementation, the iterative refinement loop also contains explicit `detach()` operations on some state variables, so the training graph is end-to-end but kept computationally manageable rather than left to grow without bound across all refinement iterations.

Algorithm 2 One RAMP-VO Training Step

```
sample one training sequence of images, event stacks, and ground-truth poses
run the full RAMP-VO forward pass with several internal refinement iterations
obtain the predicted trajectory / patch outputs
compute the training loss from these outputs
call loss.backward() to compute gradients
update the network weights with the optimizer
```

Very simple training-step pseudo-algorithm.

A.6 Training Losses

The paper states that RAMP-VO is trained end-to-end, but it does not spell out the loss in the main text. The official training code defines the loss as a weighted sum of a patch-reprojection term and a relative-pose term, accumulated over the intermediate trajectory estimates returned by the network.

For one optimization stage and one active edge (l, i), the code compares the predicted reprojected patch coordinates

$$\mathbf{P}'^{l,\text{pred}}_{ji} = \omega(\mathbf{T}_i, \mathbf{T}_j, \mathbf{P}^l_j) \tag{40}$$

with the corresponding ground-truth reprojected patch coordinates

$$\mathbf{P}'^{l,\text{gt}}_{ji} = \omega(\mathbf{T}^{\text{gt}}_i, \mathbf{T}^{\text{gt}}_j, \mathbf{P}^{l,\text{gt}}_j). \tag{41}$$

In the released code these are the quantities called `coords` and `coords_gt`. Let v_{li} be the corresponding validity mask. The code first forms the per-pixel patch error

$$e_{\text{pix}} = \left\| \mathbf{P}'^{l,\text{pred}}_{ji} - \mathbf{P}'^{l,\text{gt}}_{ji} \right\|_2, \tag{42}$$

then reshapes each $p \times p$ patch into p^2 candidate pixel errors and keeps the minimum valid one. Denoting the resulting per-patch error by e, the flow term is

$$\mathcal{L}_{\text{flow}} = w_{\text{flow}} \,\text{mean}(e), \tag{43}$$

where w_{flow} is the training hyperparameter called `flow_weight` in the code.

The pose term is built from relative poses. Using the same notation as in the rest of the document, let $\mathbf{T}_i$ denote the predicted camera poses and $\mathbf{T}^{\text{gt}}_i$ the ground-truth poses. The code forms all pairwise relative motions

$$\Delta\mathbf{T}^{\text{pred}}_{ij} = \mathbf{T}_i^{-1}\mathbf{T}_j, \qquad \Delta\mathbf{T}^{\text{gt}}_{ij} = (\mathbf{T}^{\text{gt}}_i)^{-1}\mathbf{T}^{\text{gt}}_j, \tag{44}$$

then measures the relative-pose error in the Lie algebra:

$$\mathbf{e}^{\text{pose}}_{ij} = \text{Log}\Big(\Delta\mathbf{T}^{\text{pred}}_{ij}(\Delta\mathbf{T}^{\text{gt}}_{ij})^{-1}\Big). \tag{45}$$

This error is split into translation and rotation parts,

$$t_{ij} = \left\| \mathbf{e}^{\text{pose}}_{ij,1:3} \right\|_2 , \qquad r_{ij} = \left\| \mathbf{e}^{\text{pose}}_{ij,4:6} \right\|_2 , \tag{46}$$

and the pose term is

$$\mathcal{L}_{\text{pose}} = w_{\text{pose}} \left(\text{mean}(t_{ij}) + \text{mean}(r_{ij}) \right) , \tag{47}$$

where w_{pose} is the hyperparameter called `pose_weight`.

The total training loss used in the code is therefore

$$\mathcal{L} = \sum_k \left(\mathcal{L}^{(k)}_{\text{flow}} + \mathcal{L}^{(k)}_{\text{pose}} \right) , \tag{48}$$

with the dependency structure

$$\mathcal{L}_{\text{flow}} = \mathcal{L}_{\text{flow}} \big(\{\Delta_{li}\}_{(l,i)\in\mathcal{E}} \big) , \qquad \mathcal{L}_{\text{pose}} = \mathcal{L}_{\text{pose}} \big(\{\mathbf{T}_i\}, \{\Delta_{li}, \sigma_{li}\}_{(l,i)\in\mathcal{E}} \big) , \tag{49}$$

where

$$(\Delta_{li}, \sigma_{li}) = F(\mathbf{P}'^{l}_{ji}, \mathbf{c}_i, \mathbf{m}_i, \mathbf{m}_j), \qquad (\mathbf{m}_j, \mathbf{c}_j) = K(\{\mathbf{\Sigma}^s_j\}^2_{s=0}). \tag{50}$$

where k indexes the successive intermediate trajectory estimates produced during the iterative refinement.

- $\mathcal{L}_{\text{flow}}$: supervises local patch correspondences; it directly constrains Δ_{li} and indirectly constrains $\mathbf{m}_j, \mathbf{c}_j$.
- $\mathcal{L}_{\text{pose}}$: supervises the global camera trajectory after bundle adjustment.

Only pose loss would give weak and indirect supervision to local patch matching, while only flow loss could give good local matches without the best global motion. In the released training code, the initial "structure-only" phase uses only $\mathcal{L}_{\text{flow}}$ for the first 1000 training steps. This description comes from the released training code, not from an explicit formula in the paper.

B Appendix: performance discussion

B.1 Computational Performance Discussion

The paper argues that RAMP-VO is designed to be computationally efficient at the encoder level because the RAMP encoder is highly parallelizable and avoids the more expensive sequential ConvGRU-style processing used by RAM-Net-like architectures. The strongest explicit evidence given in the paper is the timing comparison for feature extraction: on a Quadro RTX 8000 GPU, RAM Net takes 370 ms on average to extract features from a single frame or event stack, while the proposed RAMP encoder takes 47 ms, which the authors report as an 8× speedup.

At the same time, the overall method is not obviously cheap in an absolute sense. The system still combines multi-scale recurrent states, asynchronous event/image processing, learned patch refinement, and a differentiable bundle-adjustment backend. Moreover, the paper reports a substantial training cost: 350,000 training steps on sequences of 15 images and 30 event stacks, with roughly 8 days of training on a Quadro RTX 8000 GPU.

What the paper does *not* provide is a detailed audit of full-system computational cost or memory usage. In particular, it does not report GPU memory consumption, parameter counts, FLOP counts, or a full end-to-end runtime breakdown separating the encoder, the update operator, and the BA backend.

The safest conclusion is the following: the paper explicitly shows that the RAMP encoder is much faster than a RAM-Net-like alternative, but neither the paper nor the released code provides a complete end-to-end compute or memory audit.

B.2 Memory Usage Estimate

Using the official implementation, one can estimate the persistent inference-time GPU buffers. With the default values $N = 2048$ buffered frames, $M = 80$ patches per frame, memory length $\texttt{mem} = 32$, resolution 640×480, patch size $P = 3$, and mixed precision enabled for the cached feature tensors, the persistent GPU buffers account for roughly

$$187 \text{ MiB}. \tag{51}$$

The dominant term is the cached full-resolution feature tensor `fmap1_`, which alone occupies about 150 MiB.

To account for model parameters, one can use the released checkpoint sizes as a practical proxy. The official multi-scale checkpoint is 88,269,981 bytes, i.e.

$$84.2 \text{ MiB}, \tag{52}$$

while the official single-scale checkpoint is 41,239,053 bytes, i.e.

$$39.3 \text{ MiB}. \tag{53}$$

Adding these to the persistent buffer estimate gives the following lower-bound inference-time memory estimates:

$$\text{RAMP-VO multi-scale} \approx 187 + 84.2 = 271.6 \text{ MiB}, \tag{54}$$

$$\text{RAMP-VO single-scale} \approx 187 + 39.3 = 226.8 \text{ MiB}. \tag{55}$$

These numbers are only estimates and should be read as lower bounds. They do not include temporary activations inside the encoder and update operator, correlation volumes, BA workspaces, CUDA runtime overhead, or any training-time storage for gradients and optimizer states. Therefore the actual memory needed to run the full system is higher than these values, potentially by a substantial margin.